# Handbook of Modern AI Agents & Vector Architecture

*A Technical Reference on RAG, Vector Search, and Autonomous Agents*

## 1. Executive Summary & Core Workflow

Modern intelligent chatbots rely on Retrieval-Augmented Generation (RAG) to ground large language models in verifiable facts. The end-to-end RAG workflow follows a ten-stage pipeline: (1) Document Acquisition, (2) Text Extraction via PDF parsers, (3) Contextual Text Chunking, (4) High-dimensional Vector Embedding, (5) Vector Store Indexing with FAISS, (6) Natural Language Question Intake, (7) Semantic k-Nearest Neighbor Retrieval, (8) Grounded Context Synthesis into Prompts, (9) LLM Reasoning & Token Generation, and (10) Interactive State-Preserving Chat UI.

## 2. Document Parsing with PyPDF

PDF extraction extracts textual streams from diverse layout streams. PyPDFLoader processes pages sequentially, retaining document metadata including file names, page numbers, and stream bounding coordinates. Handling PDF fonts and character encodings such as UTF-8 ensures that specialized symbols and numerical figures are preserved intact.

## 3. Semantic Chunking Strategies

Text chunking determines the granularity of semantic memory. A RecursiveCharacterTextSplitter decomposes text using a hierarchy of separators: double newlines, single newlines, spaces, and punctuation. For standard technical documentation, the recommended chunk size is 1,000 characters with an overlap of 200 characters. Overlapping chunks ensure that definitions, equations, and relationships spanning paragraph boundaries remain coherent during vector similarity retrieval.

## 4. Vector Embeddings and Dense Representations

Embeddings map arbitrary text fragments into continuous dense vector spaces where semantic similarity corresponds to geometric proximity. Models such as OpenAI's text-embedding-3-small project text into a 1,536-dimensional vector space. In this space, cosine similarity measures the directional alignment of concepts. DeepSeek and OpenAI language models can then digest retrieved chunks with high precision.

## 5. Vector Indexing with FAISS

Facebook AI Similarity Search (FAISS) is an industry-standard library for indexing dense vectors. It supports multiple index types, including IndexFlatL2 for exact Euclidean distance search and IndexHNSW for approximate nearest neighbors at massive scale. For interactive PDF chatbots, FAISS delivers sub-5-millisecond query retrieval over thousands of document chunks.

| Component | Technology | Key Parameter / Metric |
|---|---|---|
| PDF Parser | PyPDFLoader / pypdf | Extracts text and page metadata |
| Chunker | RecursiveCharacterTextSplitter | Size: 1000 chars, Overlap: 200 chars |
| Embeddings | OpenAI text-embedding-3-small | 1536 Dimensions, Normalized Cosine |
| Vector Database | FAISS In-Memory Index | L2 distance / Inner Product top-k |
| LLM Providers | OpenAI (GPT-4o-mini) & DeepSeek | Grounded prompt answering |
| User Interface | Streamlit | Stateful chat with streaming tokens |

## 6. Grounded Prompt Engineering and Hallucination Mitigation

A critical advantage of RAG over vanilla LLMs is hallucination suppression. By configuring system instructions to strictly constrain answers to the retrieved document snippets, the agent produces reliable, auditable answers with source citations. If information is absent from the retrieved context, the agent politely states that the document does not contain the answer rather than guessing.